Travaux Pratiques de Programmation Android

TP n° 1

Les objectifs de ce TP sont :
- L'installation et la configuration des outils qui vont être utilisés tout au long du module ;
- La découverte d'une première application Android ;
- Développement d'une application Android simple.

Les TPs sont rédigés pour être utilisés avec Android Studio.

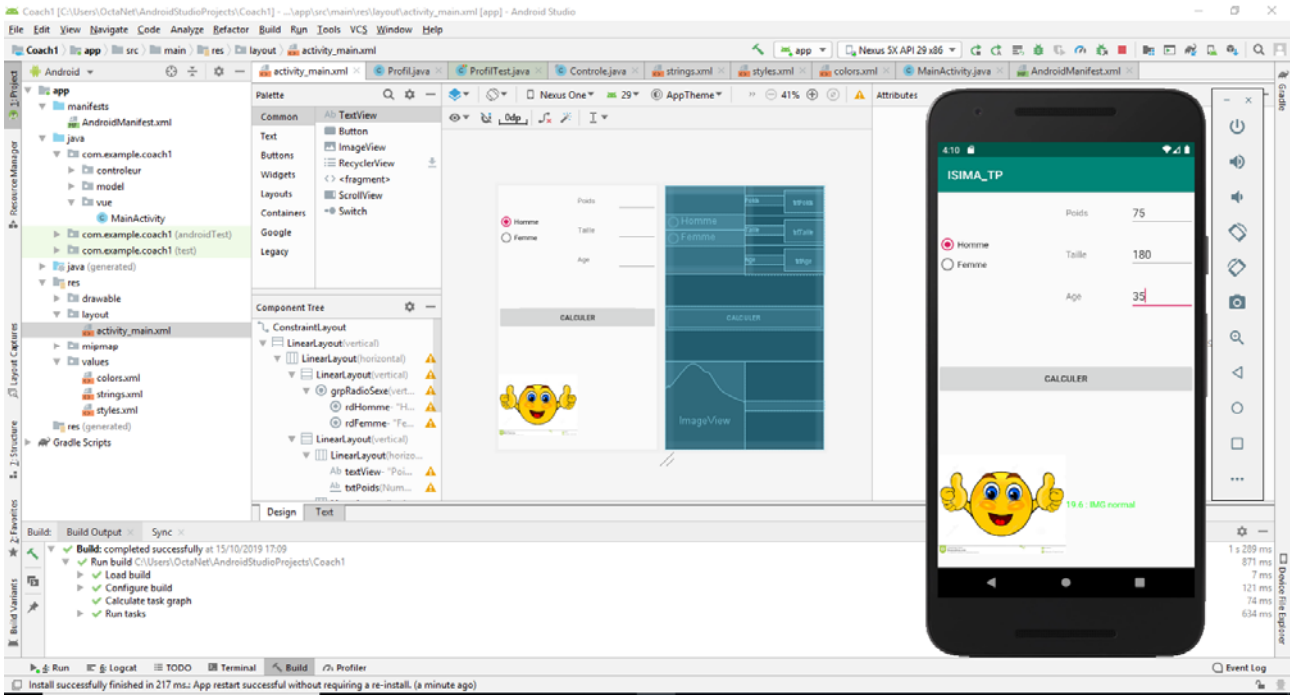


Figure 1– Vue de l'IDE Android Studio lors du développement d'un projet

A. Installation sur une machine personnelle

a. Télécharger Android Studio (https://developer.android.com/studio) pour votre système d'exploitation.
   - Sous Windows ou MacOS, double-cliquer sur le fichier téléchargé.
   - Sous Linux, décompresser l'archive, puis lancer le studio en utilisant le script androidstudio/bin/studio.sh.

b. Avancer dans le wizard. Choisir une installation « Custom », avec comme thème IntelliJ. Sélectionner tous les composants à installer.

c. À la fin du wizard, choisir « Configure/SDK Manager ». Par défaut, l'API installée est la plus récente, l'API 35. Nous allons également installer l'API 27 afin de pouvoir installer nos applications sur un maximum de plateformes utilisées actuellement, et en particulier nos tablettes.

B. Environnement utilisé

Voici quelques questions et remarques sur l'environnement utilisé :

a. Définir le JRE et le JDK. Quelle version du JDK est disponible sur votre machine ?

b. Le SDK (Software Development Kit) est une API Java offrant un ensemble de classes pour le développement d'applications Android. En constante évolution, il existe différentes versions, chacune étant compatible avec les versions précédentes (dans la majorité des cas). Une application destinée au

grand public doit être compatible avec les versions les plus utilisées au moment de sa diffusion, afin d'atteindre le plus large public possible. C'est pourquoi, en plus d'utiliser une API récente, nous choisissons également une version plus ancienne.

C. Première application

a. Choisir « File → New Project ».

b. Choisir comme cible « Empty Views Activity ».

c. Choisir le nom que vous souhaitez lui donner (par la suite, nous l'appellerons « TP0 ») et l'endroit où vous souhaitez l'enregistrer

d. Choisir Java comme étant le langage de développement.

e. Choisir l'API 27 (Oreo 8.1) comme SDK minimum.

D. Création d'un émulateur

Un émulateur sert à émuler un appareil Android particulier, afin de développer une application pour divers types d'appareil (téléphone ou tablettes, avec différentes tailles d'écran...) sans pour autant avoir besoin de tous les posséder. Dans le cadre de ce module, l'émulateur pourra vous servir également à travailler lorsque vous n'avez pas accès aux téléphones. Pour cela, nous allons créer un émulateur dont ayant des caractéristiques similaires à celles d'un téléphone.

a. Cliquer sur l'icône avec un rectangle violet « AVD Manager » (pour « Android Virtual Device »);

b. Choisir « Create Virtual Device... », puis choisir parmi les phones l'architecture prédéfinie la plus proche de votre téléphone (ou à défaut, un Nexus 5X)

c. Lancer l'émulateur ;

E. Découverte de l'application

Regarder ce que fait l'application. Vous l'aurez constaté, pour l'instant, elle se contente d'afficher « Hello world! ». Nous allons présenter les différents composants de cette application, qui feront ensuite chacun l'objet de différents TPs.

a. Dans le menu à gauche du studio, déplier le répertoire « app ». Il contient trois sous-répertoires :
   - Le répertoire manifests contient les fichiers décrivant l'architecture de l'application ;
   - Le répertoire java contient les fichiers source Java ;
   - Le répertoire res contient tout ce qui a trait à l'interface graphique de l'application.

b. Consulter le fichier AndroidManifest.xml du répertoire manifests. Il décrit l'architecture de l'application : son nom, son thème, les activités qu'elle contient (il n'y en a pour l'instant qu'une). Ce fichier doit être édité lorsqu'on ajoute de nouvelles activités, à la main ou automatiquement via l'utilisation des fonctionnalités d'Android Studio.
   - Consulter le fichier MainActivity.java du répertoire java. Il s'agit d'une classe Java implantant l'activité, contenant la méthode onCreate. Cette methode est appelée à la création de l'activité. Un de ses rôles est de démarrer l'interface graphique, via l'appel à la méthode setContentView ;

c. Ouvrir le fichier activity_main.xml du répertoire res/layout. Il décrit l'interface graphique de l'activité. Android Studio propose deux vues pour lire ce fichier :
   - L'onglet « Split » ou « code » affiche simplement le contenu du fichier, à savoir du XML décrivant les diverses couches de l'interface, ainsi qu'une prévisualisation du résultat (pour laquelle on peut choisir différents appareils, notamment notre émulateur);

- L'onglet « Design » affiche également la prévisualisation du résultat et permet facilement d'ajouter de nouveaux éléments par glisser-déposer.

Regarder le contenu XML : l'interface contient une seule couche de type RelativeLayout, elle-même contenant une zone de texte TextView contenant la chaîne de caractères « Hello world! ». En passant le curseur sur cette chaîne, on constate qu'elle est en réalité définie dans la variable @string/hello_world, que nous allons voir maintenant.

Cette variable est définie dans le fichier strings.xml du répertoire res/values, ainsi que le nom de l'application, et une variable contenant l'unique élément du menu actuel. Le fait de définir ainsi des variables contenant les chaînes de caractères apparaissant dans l'interface permet de gérer facilement les différentes localisations de l'application (version anglaise, française, ... etc), comme nous le verrons par la suite.

Exercice 1

Modifier l'application de manière à ce qu'elle affiche « This is my first Android app. » au lieu de « Hello world! » tout en choisissant "Lineaire Layout" comme gestionnaire de présentation.

Exercice 2

Développer une application Android qui affiche "*Soyez les bienvenus à l'FSM*".

a. Modifier la police du texte
b. Centrer le texte
c. Modifier la couleur de fond
d. Modifier la couleur de la police
e. Changer la taille du texte
f. Ajouter un Boutton de commande "Valider"

Exercice 3

Dans la classe MainActivity.java ajouter un Toast à afficher en bas de l'activité à travers le code suivant :

```
Toast.makeText(this, "Bonjour mes Amis", Toast.LENGTH_LONG).show();
```

Exercice 4

Dans la classe MainActivity.java ajouter un message à afficher dans la fenêtre LogCat. Localiser ce message après exécution de l'application.

Exemple : `Log.i("Test","salut...!!!");`

Exercice 5

Développer le formulaire de l'application présenté par la figure 2 de la page 4, en utilisant le XML.

Exercice 6

Développer dans le même projet le formulaire de l'application présenté par la figure 3 de la page 4, en utilisant le XML.

F. Attribution d'écouteurs aux boutons

Android offre deux possibilités :

- Soit vous définissez l'attribut android:onClick et vous créez une méthode dans l'activité concernée.
- Soit vous définissez l'activité elle-même en tant qu'écouteur, et voici la démarche à faire : Il faut éditer la méthode onCreate de l'activité concernée et rajouter des instructions comme celles-ci.

Button btn1 = findViewById(R.id.btn1);

btn1.setOnClickListener(this);

Cela consiste à attribuer *this* en tant qu'écouteur des clics sur le bouton concerné.
AndroidStudio vous signalera que l'activité *this* n'est pas un écouteur correct. Il suffit simplement qu'elle implémente l'interface *OnClickListener*.
Ensuite, AndroidStudio signalera qu'il manque la méthode exigée par l'interface *OnClickListener* c'est à dire *onClick(View v)*. Son paramètre est une vue v d'un type général View, mais en réalité, ce sera un Button.

Exercice 7

Développer l'application Android de la figure 3 dont le formulaire a été développé dans l'exercice 6. Au lancement de l'application le formulaire invite l'utilisateur à saisir son nom. En appuyant sur le bouton Valider, la dernière zone de texte affiche bonjour suivi du nom de l'utilisateur. En plus un Toast est affiché en bas signalant le message "c'est ok". Le clique sur le bouton "Quitter" permet de fermer l'application. La figure 4 illustre un exemple d'exécution.



Figure 2.

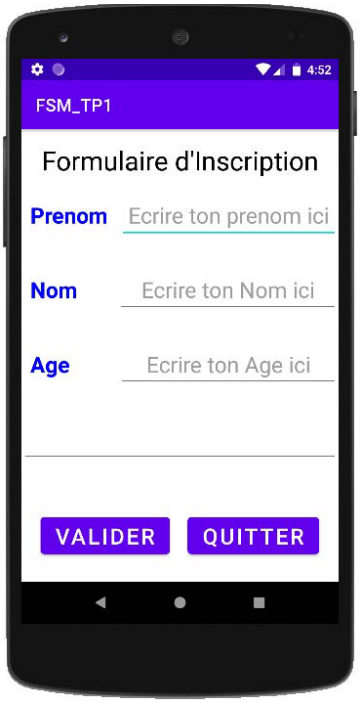


Figure 3.

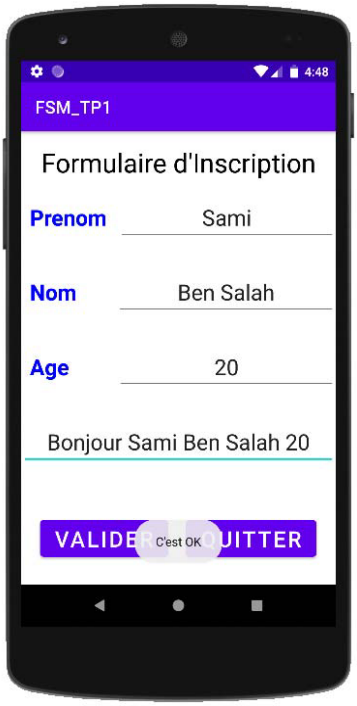


Figure 4.